

# LAPORAN PRAKTIKUM KONTROL CERDAS WEEK 4

Nama : Ryamizard Agil Briantama  
NIM : 224308095  
Kelas : TKA-7D  
Akun Github : <https://github.com/RyamizardAgilBriantama3>

## 1. Judul Percobaan: *Reinforcement Learning for Autonomous Control*

## 2. Tujuan Percobaan:

Tujuan dari percobaan Deep Learning for Intelligent Control Systems adalah :

- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.

## 3. Landasan Teori:

Kecerdasan buatan atau Artificial Intelligence (AI) adalah sistem yang mempelajari bagaimana membuat komputer dapat berpikir, belajar, dan bertindak seperti manusia (Dosari & Abouellail, 2023). Reinforcement Learning (RL) adalah metode pembelajaran di mana agen belajar mengambil keputusan melalui interaksi dengan lingkungan. Agen memperoleh reward atau penalty sebagai umpan balik dari tindakannya, sehingga ia dapat mengoptimalkan strategi untuk mencapai tujuan tertentu. Pada dasarnya memungkinkan mesin atau agen mempelajari tindakannya berdasarkan timbal balik dari lingkungan sehingga dapat beradaptasi dengan lingkungan seiring dengan berjalannya waktu. Prinsip yang digunakan dalam metode ini didasarkan pada teori reinforcement yang pada intinya adalah ‘konsekuensi mempengaruhi tindakan’. Ada 3 prinsip dasar pada teori reinforcement yakni

konsekuensi yang berakibat baik mendorong terjadinya tindakan, konsekuensi yang berakibat buruk mendorong berkurangnya tindakan dan konsekuensi yang tidak ada dampaknya tidak memengaruhi tindakan (t.t.).

Deep Q-Network (DQN) adalah algoritma pembelajaran penguatan yang revolusioner, menggabungkan kekuatan Q-Learning dan Deep Learning untuk mengatasi keterbatasan Q-Learning tradisional. Q-Learning, yang berfokus pada pembelajaran kebijakan optimal melalui Q-table, menjadi tidak efisien dalam lingkungan dengan state yang kompleks atau kontinu. DQN mengatasi masalah ini dengan menggunakan jaringan saraf tiruan untuk memperkirakan nilai Q, memungkinkan penanganan input berdimensi tinggi seperti gambar dan state yang sangat banyak. Dengan teknik seperti experience replay dan target network, DQN meningkatkan stabilitas dan efisiensi pembelajaran, menjadikannya alat yang ampuh untuk berbagai aplikasi seperti permainan video, robotika, dan mobil otonom (Purba dkk., 2022).

Penerapan Reinforcement Learning (RL) telah berkembang menjadi untuk sistem kontrol otonom yang cerdas dan adaptif. Penerapannya mencakup berbagai bidang, mulai dari kendaraan otonom yang dilatih untuk mengambil keputusan navigasi kompleks dalam kondisi lalu lintas dinamis, hingga robotika yang memungkinkan robot melakukan tugas-tugas sulit seperti manipulasi objek dan navigasi di lingkungan yang tidak terstruktur. Reinforcement Learning (RL) juga digunakan untuk mengoptimalkan sistem kontrol industri, memungkinkan adaptasi terhadap perubahan kondisi dan peningkatan efisiensi, serta dalam kontrol UAV untuk tugas-tugas seperti pengawasan dan pemetaan, di mana UAV belajar terbang secara otonom di lingkungan yang kompleks.

Software yang digunakan dalam percobaan ini berupa Python dan Open CV. OpenCV (Open Source Computer Vision Library) adalah perpustakaan perangkat lunak yang dirancang khusus untuk pengolahan citra dan visi komputer. Dikembangkan oleh Intel dan sekarang bersifat open-source, OpenCV menyediakan berbagai alat dan fungsi yang memudahkan pengembang untuk membuat aplikasi berbasis pengolahan citra dan visi komputer. Python, sebagai salah satu bahasa pemrograman yang didukung

oleh OpenCV, menjadi pilihan populer karena sintaksnya yang mudah dipahami dan kemampuannya untuk integrasi dengan berbagai pustaka lain, seperti NumPy, SciPy, dan scikit-learn. Dengan OpenCV Python, pengembang dapat mengakses berbagai algoritma canggih untuk tugas-tugas seperti deteksi objek, pelacakan, pengenalan wajah, dan segmentasi citra. Sehingga bahasa pemrograman Python yang diprogram memiliki keahlian yaitu memberikan hasil keluaran sesuai input dari dataset, metode dan algoritma yang digunakan (Zebua & Rosyani, 2024).

#### **4. Analisis dan Diskusi:**

- **Analisis**

Pada praktikum minggu keempat ini percobaan yang dilakukan adalah mengimplementasikan agen Reinforcement Learning (RL) menggunakan algoritma Deep Q-Network (DQN). Tahap awal yang dilakukan adalah membuat file `dqn_agent.py` yang kemudian diisikan kode-kode program untuk implementasi Reinforcement Learning (RL) menggunakan algoritma Deep Q-Network (DQN). Pertama mendefinisikan sebuah kelas bernama `DQNAgent` dengan kode `class DQNAgent`: yang berfungsi membuat kode program untuk mengimplementasikan agen pembelajaran penguatan Reinforcement Learning (RL) yang menggunakan algoritma Deep Q-Network (DQN). Selanjutnya, didalam kelas tersebut terbagi menjadi beberapa bagian antara lain `def __init__(self, state_size, action_size)`: adalah sebuah konstruktor kelas, yang dipanggil saat objek `DQNAgent` dibuat. Fungsinya yaitu untuk menginisialisasi atribut-atribut penting agen, seperti ukuran keadaan (`state_size`) dan ukuran tindakan (`action_size`), serta membangun model jaringan saraf tiruan yang akan digunakan oleh agen. Lalu, `def build_model(self)`: adalah metode yang bertanggung jawab untuk mendefinisikan arsitektur jaringan saraf tiruan yang akan digunakan untuk memperkirakan nilai Q. Kemudian, `def remember(self, state, action, reward, next_state, done)`: adalah metode yang digunakan untuk menyimpan pengalaman agen ke dalam memori replay. Pengalaman ini terdiri dari keadaan saat ini, tindakan yang diambil, imbalan yang diterima,

keadaan berikutnya, dan apakah episode selesai. Kemudian, `def act(self, state):` adalah metode yang menentukan tindakan apa yang akan diambil oleh agen berdasarkan keadaan saat ini. Metode ini biasanya menggunakan strategi epsilon-greedy untuk menyeimbangkan eksplorasi dan eksploitasi. Terakhir, `def replay(self, batch_size):` adalah metode yang melatih jaringan saraf tiruan dengan mengambil sampel acak dari memori replay. Metode ini menghitung kerugian (loss) dan memperbarui bobot model menggunakan algoritma optimisasi seperti Adam. Secara keseluruhan, metode-metode ini bekerja sama untuk memungkinkan agen DQN belajar dan meningkatkan kinerjanya dalam lingkungan yang diberikan.

Kedua, menentukan apakah sebuah skrip dijalankan sebagai program utama atau diimpor sebagai modul dengan menggunakan kode `if __name__ == '__main__':` yang berfungsi memastikan bahwa kode pelatihan hanya dijalankan ketika skrip dijalankan secara langsung. Kemudian, membuat kode program untuk memulai perulangan yang akan mengulangi blok kode di dalamnya sebanyak jumlah episode yang diinputkan. Dengan menggunakan kode `for e in range(episode):` berfungsi untuk melatih agen selama beberapa episode, yang memungkinkan agen untuk belajar dan meningkatkan kinerjanya seiring waktu.

Tahap selanjutnya yang dilakukan adalah membuat file `test_agent.py` untuk menguji agen yang telah dilatih pada environment. Kemudian, lingkungan `CartPole-v1` dibuat, dan ukuran state dan action diidentifikasi untuk menginisialisasi agen DQN. Agen DQN diinisialisasi dengan epsilon yang sangat kecil untuk meminimalkan eksplorasi dan memaksimalkan eksploitasi, yang berarti agen akan cenderung memilih tindakan berdasarkan apa yang telah dipelajarinya. Program kemudian menjalankan episode pengujian sesuai jumlah yang diinputkan. Di setiap episode, lingkungan direset, state awal direshape, dan agen berinteraksi dengan lingkungan selama maksimal langkah yang diinputkan. Di setiap langkah, lingkungan dirender, agen memilih tindakan berdasarkan state saat ini, dan lingkungan memberikan state berikutnya, reward, dan status selesai. State berikutnya direshape untuk input agen. Jika episode selesai

(CartPole jatuh), skor episode dicetak, dan episode berakhir. Setelah semua episode pengujian selesai, lingkungan ditutup.

Kemudian, dilakukan modifikasi program dengan menambahkan teknik target network untuk meningkatkan stabilitas pelatihan program serta menguji agen pada environment lain seperti mountaincar-v0 untuk dibandingkan performanya. Untuk modifikasi yang pertama adalah dengan menambahkan visualisasi grafis dari lingkungan CartPole-v1 dengan kode `gym.make('CartPole-v1', render_mode='rgb_array')` menggunakan Pygame dengan kode `def init_pygame(): | def render_pygame` dan menampilkan grafik score dan nilai epsilon secara real time menggunakan matplotlib dengan kode `plt.ion() | plt.grid() | plt.ioff() | plt.show()`. Untuk modifikasi yang kedua adalah meningkatkan stabilitas pelatihan program pada environment MountainCar-v0. Menggunakan kode program yang kurang lebih sama dengan kode program modifikasi CartPole-v1 yang membedakan adalah lingkungan permainan pada modifikasi ini diubah dengan MountainCar-v0 yang bertujuan untuk menggerakkan mobil ke puncak bukit menggunakan kode `gym.make('MountainCar-v0', render_mode='rgb_array')`, menambahkan kode `steps_list` dan menampilkan jumlah langkah dalam grafik, menambahkan kode `update_plot()`: memiliki fungsi untuk mengupdate plot secara dinamis. Pada modifikasi menggunakan MountainCar-v0 juga ditambahkan visualisasi grafik yang lebih lengkap dengan menampilkan jumlah langkah per episode.

Perbedaan hasil yang ditampilkan oleh kedua kode antara lain pada program CartPole-v1 karakteristiknya memiliki ruang keadaan dan tindakan yang relatif sederhana, dimana hasil yang diperoleh adalah agen belajar dengan cepat untuk menjaga tiang tetap tegak dalam waktu yang lama. Sedangkan program MountainCar-v0 karakteristiknya memiliki ruang keadaan dan tindakan yang lebih kompleks, dengan tantangan mencapai tujuan, dimana hasil yang diperoleh adalah agen memerlukan lebih banyak episode dan eksplorasi untuk belajar mencapai tujuan. Perbedaan waktu yang dibutuhkan untuk menjalankan kedua program

tersebut dapat dipengaruhi oleh berbagai faktor, baik dari sisi perangkat keras (hardware) maupun perangkat lunak (software). Dari sisi perangkat keras (hardware) antara lain prosesor (CPU), memori (RAM), dan kartu grafis (GPU). Sedangkan dari sisi perangkat lunak (software) antara lain sistem operasi, optimasi kode, dan bahasa pemrograman yang digunakan.

- **Diskusi**

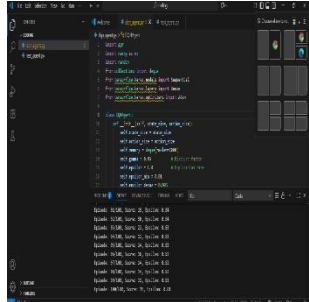
Metode implementasi agen Reinforcement Learning (RL) memiliki perbedaan mendasar dengan supervised learning dalam konteks sistem kendali terletak pada jenis data, tujuan, metode pembelajaran, interaksi, dan aplikasi. Supervised learning mengandalkan data berlabel untuk memprediksi output, sementara Reinforcement Learning (RL) berinteraksi dengan lingkungan dinamis dan belajar dari reward. Tujuan supervised learning adalah prediksi, sedangkan Reinforcement Learning (RL) adalah optimasi tindakan untuk reward maksimum. Strategi eksplorasi dan eksploitasi dalam Reinforcement Learning (RL) dapat dioptimalkan dengan teknik seperti Epsilon-greedy policy, Boltzmann Exploration, Upper Confidence Bound (UCB), dan Thompson Sampling. Teknik-teknik ini menyeimbangkan antara mencoba tindakan baru dan memanfaatkan strategi yang telah terbukti, memungkinkan agen Reinforcement Learning (RL) belajar lebih efisien dan menemukan kebijakan optimal.

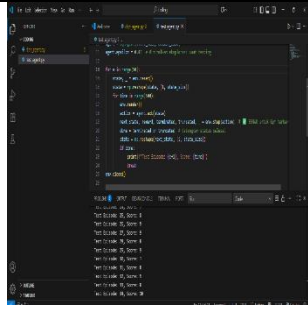
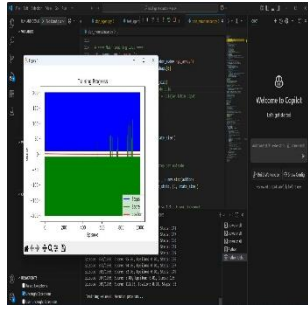
## **5. Assignment:**

Dalam praktikum yang telah dilakukan mengenai modifikasi program dengan menambahkan teknik target network untuk meningkatkan stabilitas pelatihan program serta menguji agen pada environment lain seperti mountaincar-v0 untuk dibandingkan performanya. Dengan mengimplementasikan agen Reinforcement Learning (RL) menggunakan algoritma Deep Q-Network (DQN) dan menambahkan visualisasi grafis dari lingkungan CartPole-v1 serta meningkatkan stabilitas pelatihan program pada environment MountainCar-v0 sistem memiliki kelebihan pada visualisasi real-time menggunakan Pygame, yang memberikan pemahaman intuitif tentang interaksi agen dengan lingkungan, serta plotting grafik dinamis dengan matplotlib yang memantau kemajuan pelatihan. Penggunaan Keras dan

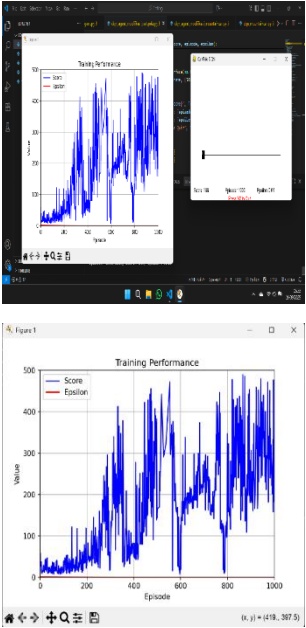
TensorFlow mempermudah pembangunan dan pelatihan model jaringan saraf tiruan, sementara memori replay dan strategi epsilon-greedy meningkatkan stabilitas dan efisiensi pembelajaran. Namun, kode ini juga memiliki kekurangan, yaitu kompleksitas visualisasi yang dapat memperlambat pelatihan dan menambah beban komputasi, ketergantungan pada pustaka eksternal yang beragam, dan waktu pelatihan yang relatif lama, terutama untuk lingkungan yang kompleks seperti MountainCar-v0

## 6. Data dan Output Hasil Pengamatan:

| No | Variabel                                          | Hasil Pengamatan                                                                                                                                                                                                                                                                                                                              | Output                                                                               |
|----|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 1  | Menggunakan program dqn_agent.py tanpa modifikasi | Pada Percobaan mengimplementasikan agen Reinforcement Learning (RL) menggunakan algoritma Deep Q-Network (DQN) dengan parameter episode=100 dan batch_size =32 memerlukan waktu kurang lebih 1 jam. Hal ini disebabkan kompleksitas algoritma DQN itu sendiri dan jumlah episode dan ukuran batch yang digunakan juga mempengaruhi pelatihan. |  |
| 2. | Menggunakan program test_agent.py                 | Pada tahap ini berfungsi untuk menguji agent yang                                                                                                                                                                                                                                                                                             |                                                                                      |

|    |                                                     |                                                                                                                                                                                                                                                                          |                                                                                       |
|----|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|    |                                                     | <p>telah dilatih dengan pengujian episode 50 memerlukan waktu lebih singkat sekitar 30 menit. Hal ini disebabkan karena jumlah episode pengujian yang lebih sedikit, secara tidak langsung dapat mengurangi total waktu yang dibutuhkan untuk menjalankan pengujian.</p> |    |
| 3. | <p>Menggunakan program dqn_agent_MountainCar.py</p> | <p>Pada percobaan ini menggunakan MountainCar dengan parameter episode 1000 dan batch size 32 membutuhkan waktu kurang lebih 6 jam untuk menjalankan program hingga episode 1000. Hal ini disebabkan oleh kompleksitas lingkungan dan model yang digunakan.</p>          |  |



|           |                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                     |
|-----------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <p>4.</p> | <p>Menggunakan program dqn_agent_modifikasi Cartpole.py</p> | <p>Pada percobaan menambahkan grafisdari lingkungan menggunakan CartPole-v1 dengan parameter episode 1000 dan batch_size 32 membutuhkan waktu sekitar 11 jam untuk menjalankan program hingga 1000 episode. Hal ini disebabkan visualisasi real-time menggunakan Pygame memerlukan daya komputasi yang signifikan untuk merender setiap frame, memperlambat setiap iterasi pelatihan dan banyak faktor lainnya.</p> |  |
|-----------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|

## **7. Kesimpulan:**

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat ditarik kesimpulan yaitu:

- 1) Reinforcement Learning (RL) memungkinkan agen untuk belajar mengambil keputusan melalui interaksi dengan lingkungan untuk mencapai tujuan tertentu.
- 2) Algoritma Deep Q-Network (DQN) berhasil diimplementasikan untuk melatih agen dalam lingkungan simulasi OpenAI Gym, dengan penambahan visualisasi Pygame dan plotting Matplotlib untuk pemantauan kinerja.
- 3) Hasil dari modifikasi program menunjukkan bahwa parameter dan lingkungan yang berbeda mempengaruhi performa agen, dengan CartPole-v1 menunjukkan pembelajaran yang lebih cepat dibandingkan MountainCar-v0 yang lebih kompleks.
- 4) Praktikum ini memberikan pemahaman mendalam tentang pentingnya eksplorasi dan eksploitasi dalam RL, serta potensi aplikasinya dalam sistem kendali otonom yang adaptif.

## **8. Saran:**

Untuk meningkatkan performa sistem, sebaiknya menambahkan opsi penyimpanan dan pemuatan model, menerapkan teknik DQN yang lebih canggih, menyediakan penyesuaian parameter melalui argumen baris perintah, menambahkan interaksi pengguna yang lebih baik, serta memberikan opsi untuk mengatur ukuran memori replay. Meningkatkan kompleksitas simulasi dengan lingkungan yang lebih realistis atau penerapan pada perangkat keras fisik seperti robot otonom akan memberikan wawasan lebih lanjut mengenai kesiapan algoritma RL dalam skenario dunia nyata. Dengan menerapkan saran ini, praktikum diharapkan dapat memberikan pemahaman yang lebih mendalam tentang strategi optimasi RL dan aplikasinya dalam sistem kendali otonom.

## 9. Daftar Pustaka:

- Admin,+C007C014++Azhari+penerapan+model+reinforcement+learning+  
bahan+baku+produksi . (t.t.).
- Dosari, F. H. M. A., & Abouellail, S. I. A. D. (2023). Artificial Intelligence (AI) Techniques for Intelligent Control Systems in Mechanical Engineering. *American Journal of Smart Technology and Solutions*, 2(2), 55–64. <https://doi.org/10.54536/ajsts.v2i2.2188>
- Purba, R., Lestari, W. S., & Ulina, M. (2022). Deteksi Serangan DDoS Menggunakan Deep Q-Network. 9(1).
- Zebua, E. T. P., & Rosyani, P. (2024). Perancangan Deteksi Objek Kendaraan Bermotor Berbasis OpenCV Python menggunakan Metode HOG-SVM untuk Analisis Lalu Lintas Cerdas. 2(1).